

Ultimate React.js Interview Questions Bank for 30 LPA Packages

Section 1: React Fundamentals

- What is React?
- React Architecture.
- React basic setup and advanced configurations.
- What is JSX?
- How many types of components are there in React? (Functional vs Class)
- What are stateless and stateful components in React?
- What is Rendering in React?
- Explain the virtual DOM and its benefits
- Describe the component lifecycle in both class and functional components
- What are the key differences between Redux and Context API?

- Explain how you would structure a large-scale React application
- What are controlled vs. uncontrolled components?
- How does React handle events differently from standard DOM events?
- Explain the concept of lifting state up
- What are React portals and when would you use them?
- Describe how you would test a React component
- How does React's reconciliation algorithm work?
- What is the difference between useMemo and useCallback?
- How would you implement code splitting in a React application?
- Explain the difference between client-side and server-side rendering in React
- What are higher-order components (HOCs) and how do they compare to render props?
- How does React's concurrent rendering improve performance?
- What are some common pitfalls when using useEffect?
- How would you manage forms in a React application efficiently?
- What is suspense in React, and how does it work with data fetching?
- How does React handle hydration in server-rendered applications? where i add all these quesitons
- What is Virtual DOM and how does it work?
- Flow of data in React.
- what is the memoization is react memoizationa react memoization is the same
- what is the debouncing and what is the applicaitons in the react
- what are the new hooks introduced in react v 19.

Section 2: State & Props

- States & Props.
- Difference between State and Props.
- What is `setState` in React?
- Why is state immutable in React?
- Which function is used for updating the state of a component?
- What is the advantage of choosing props over states?
- Give an example of a stateless component.
- What is Props Validation?
- How do you validate props in a React component?
- What are the different data types available in `react.PropTypes`?
- What are different types of PropTypes?

Section 3: Component Types & Patterns

- What are controlled and uncontrolled components?
- What are controlled & uncontrolled events?
- How can we directly access DOM in React? (Refs)
- What are inline functions in React?
- What is conditional rendering in React?
- How would you conditionally render a component in render function?
- What are Pure Components in React?
- What are Higher Order Components (HOC) and their uses?
- What is Prop Drilling?
- Avoiding prop drilling: Context API vs event emitters vs composition patterns.
- Building scalable component architecture (Container-Presenter pattern, compound components, render props).

Section 4: Events

- What is event delegation?
- How react handle Event
- what is synthetic event in react
- explain the event flow lifcycle
- How react handle event
- What is event bubbling and event capturing ?
- Difference Between Native Event and Synthetic Event
- Why React Uses Synthetic Events?
- How Event Delegation Works in React?
- Difference Between onClick in React and onclick in HTML?
- How to Handle Multiple Events in React?
- What are Event Pooling in React?
- What Happens When State Updates Inside Event Handler?
- Are React Events Synchronous or Asynchronous?
- How to Optimize Event Handlers in React?
- What is Passive Event Listener?
- How does batching work in React events?
- Explain priority of events in React Fiber.

Section 5: React Lifecycle & Hooks

- Explain the React component lifecycle.
- Lifecycle methods in ReactJS.
- What is the cleanup function in `useEffect`?
- Mounting vs Updating vs Unmounting.
- When will the Render method fire?
- Scenario-based questions from React Lifecycle.

- React Latest Features.
- React Hooks (useState, useEffect, useReducer, useContext).
- Concept of React Hooks.
- what is the useLayoutEffect hooks
- Cancel API request when leaving page.
- What is useLayoutEffect?
- Why do we need useLayoutEffect?
- How do you prevent an infinite loop in `useEffect`?
- How do you stop memory leaks in React?
- How do you cancel an API request when a component unmounts?

Section 6: State Management

- What is State in React?
- What are Different Types of State in React?
- Scenario: How do you decide where to keep state?
- Explain Redux Toolkit Architecture
- Why Redux Toolkit Instead of Redux?
- How will you manage authentication in a React application?
- Your dashboard has 50 components. Updating one component causes many components to re-render. How will you optimize?
- You have a table displaying 10,000 users. Updating one row re-renders the entire table. How will you solve?
- Product list is used in Navbar, Sidebar, and Product page. How will you manage it? (Scenario: Multiple Components Need Same API Data)
- Context API vs Redux Toolkit
- You used Context API but every component is re-rendering. Why? (You used Context API but every component is re-rendering. Why?)

- How do you manage state in a complex React application?
- What is Normalization in Redux?
- How will you manage chat state?
- Patterns for global state: Redux Toolkit.
- How do you manage state in a complex app to avoid unnecessary re-renders?
- What is Immer?

Section 6: Advanced React Internals

1. React Fiber Architecture & Reconciliation

Conceptual Questions

- What is React Fiber?
- Why did React introduce the Fiber architecture?
- Explain React's reconciliation algorithm.
- How does React decide which components need to re-render?
- What is the difference between reconciliation and rendering?
- How does React compare two Virtual DOM trees?
- What is the role of keys during reconciliation?
- What happens if keys are unstable?
- Why shouldn't we use array indexes as keys?
- What is the Render phase and Commit phase?
- Can React pause rendering? How?
- What is interruptible rendering?
- How does Fiber improve user experience?
- How does React schedule work internally?
- What are Fiber nodes?

2. Virtual DOM vs Real DOM

Conceptual Questions

- What is the Virtual DOM?
- What is the Real DOM?
- Why does React use a Virtual DOM?
- Does React always update the Real DOM after every state change?
- What actually triggers a re-render?
- What is the difference between re-rendering and DOM updates?
- How does React minimize DOM operations?
- Does React compare HTML strings?
- Is Virtual DOM always faster?
- What are the limitations of the Virtual DOM?

3. React Rendering, Batching & Concurrency

Conceptual Questions

- What is batching in React?
- What is automatic batching in React 18?
- How was batching different before React 18?
- What is concurrent rendering?
- Does concurrent rendering make React multithreaded?
- What is interruptible rendering?
- What are render priorities?
- What are urgent vs non-urgent updates?

- What is React Scheduler?
- How does React avoid blocking the main thread?

4. React 18 Concurrent Features

- What is `startTransition`?
- What is `useTransition`?
- What is `useDeferredValue`?
- Difference between `startTransition` and `useDeferredValue`?
- What is a transition update?
- What are urgent updates?
- How does React prioritize updates?
- How do transitions improve UX?
- Can transitions replace debouncing?
- Why uncontrolled components fix typing lag
- Why useRef doesn't cause re-render
- How Suspense improves UX
- How hydration works
- How React batches updates
- Why key must be stable
- Real case for useCallback + useMemo

5 . Suspense Internals

- What is React Suspense?
- How does Suspense work internally?
- How is Suspense different from `React.lazy()`?

- What is a Suspense boundary?
- What causes Suspense to show a fallback?
- Can Suspense work with data fetching?
- What happens when multiple Suspense boundaries exist?
- How does nested Suspense work?
- What is SuspenseList?
- How does streaming SSR use Suspense?

Section 8: Performance & Optimization

- Performance profiling: React DevTools, flame charts, avoiding wasted renders.
- Optimizing large lists — windowing using `react-window` / `react-virtualized`.
- How would you optimize a React application rendering 100k+ items in a list?
- What strategies would you use to improve page load time for a global audience?
- How React optimizes rendering with batching & concurrency features.

Section 11: React + Redux

- What is Redux?
- Why do we need Redux?
- Redux Architecture.

- Difference between Redux and Flux.
- Concept of Connect and Provider.
- How to connect React with Redux?
- What is mapStateToProps and mapDispatchToProps?
- What is store in Redux?
- What is Reducer?
- What are combined reducers?
- Concept of Redux Thunk.
- What is redux-saga?
- What is Context API? (difference with Redux).

Sections 24 : Security in React

1. **What is XSS (Cross-Site Scripting) and how do you prevent it in React?**
   - DOM sanitization, libraries like DOMPurify.
2. **What is CSRF (Cross-Site Request Forgery) and how do you mitigate it?**
   - CSRF tokens, SameSite cookies.
3. **How do you securely store JWT tokens in a React app?**
   - HttpOnly cookies vs localStorage.
4. **What are the risks of storing tokens in localStorage/sessionStorage?**
5. **How do you prevent clickjacking attacks in React apps?**
   - X-Frame-Options, CSP headers.
6. **Explain Content Security Policy (CSP) and its role in React security.**
7. **How do you handle user input safely in React forms?**
   - Avoid `dangerouslySetInnerHTML`, sanitize inputs.
8. **How do you implement secure authentication and token refresh flow in React?**

- Axios interceptors, refresh token strategy.

9. **What is the difference between HttpOnly and Secure cookies?**
10. **How do you prevent sensitive data leaks in client-side logs or error messages?**

🟢 API, Async Handling & Networking

1. How do you send multiple API requests on a single button click? **(Parallel Calls)**
2. How do you handle errors if multiple requests are sent simultaneously? **(Error Handling - Parallel)**
3. If a token is saved in localStorage, anyone can misuse it — what should you do? **(Security - Auth)**
4. Can you share multiple ways to intercept API requests in React projects? **(Interceptors / Middleware)**
5. How do you call and handle responses from AI APIs like OpenAI, Hugging Face, or Google Cloud AI? **(AI Integration)**
6. How do you handle rate-limiting or large payloads when consuming AI services? **(Scalability / API Optimization)**
7. What is WebSocket? **(Real-time Communication)**
8. what is the use of the headers in the http request **(HTTP Basics)**
9. what happend when we hit a a url in borwsers **(Browser Internals)**

🟢 Security & Best Practices

1. How do you secure your application from attacks? **(Application Security)**
2. How do you handle privacy and security for AI-powered data? **(Data Security)**

3. How do you prevent XSS (Cross-Site Scripting) in a frontend app? **(Security - XSS)**
4. How do you prevent XSS and CSRF attacks? **(Security - XSS/CSRF)**

🟢 Performance Optimization

1.

🟢 React Advanced / Application Behavior

1. How do you prevent a login page from being accessible after the user is logged in? **(Auth Guard / Routing)**
2. How do you handle API errors gracefully in the UI? **(UX / Error Handling)**
3. A component is fetching the same data multiple times unnecessarily. How do you fix it? **(Optimization / useEffect Issue)**
4. How do you handle a form with 20+ fields efficiently? **(Forms Optimization)**
5. Your React app needs to support dark mode. How do you implement it? **(UI Theming)**
6. How do you handle offline and online state gracefully in React with service workers? **(PWA / Offline Support)**
7. Production is not getting the latest deployed update unless in incognito mode. How do you solve this? **(Caching Issue)**

🟢 Browser, Rendering & Web Fundamentals

1. Difference between SSR and CSR? **(Rendering Strategies)**
2. What is Service Worker? **(PWA)**
3. What is hydration and dehydration? **(SSR Concepts)**
4. What is Critical Rendering Path? **(Browser Rendering)**
5. what is crp (criticial rendering path)? **(Browser Rendering)**

6. what event can we call use when a website is loading ? in depth overview of crp **(Browser Lifecycle)**
7. what are semantic html elements **(HTML Basics)**
8. what is srcset in html **(Responsive Images)**
9. difference betweent he display none and visibility hidden **(CSS Rendering)**
10. what is the render blocking resources **(Performance)**

🟢 SPA, Architecture & System Design

1. Why migrate from Angular to React? What challenges did you face? **(Architecture Decision)**
2. Difference between GraphQL and REST API? **(API Design)**
3. How do you decide which library to use in a project? **(Decision Making)**
4. What is Webpack and how does it create bundles? **(Build Tools)**
5. explain the webpack build process in the javascript **(Build Tools Internals)**
6. What is Suspense in React? **(Concurrent Features)**
7. What are loaders and actions? (React Router / data handling concepts) **(Routing Data APIs)**
8. What is React Flusp **(React Internals / Debugging)**
9. what are preload, reconnect , prefetch , and pre-render? **(Resource Hints)**
10. How can you do caching on website **(Caching Strategies)**
11. what are eTag , cache control and document fragment **(Caching / DOM Optimization)**
12. How do you optimize assets what is image compression what is difference between the webp, png and jpg **(Asset Optimization)**
13. what is Critical Rendering Path? **(Rendering Pipeline)**

🟢 SPA Behavior & Browser Mechanics

1. What happens behind the scenes when a Single Page Application loads? **(SPA Lifecycle)**
2. How many times does the HTML file load in an SPA vs an MPA (Multi-Page Application)? **(SPA vs MPA)**
3. If you use document.querySelector() and update a DOM element, does the page reload in traditional apps? **(DOM Manipulation)**
4. Why does navigation in SPAs not trigger a full page reload? **(Routing Mechanism)**
5. How does React update the view when the URL changes without reloading the document? **(Virtual DOM / Router)**
6. How are assets (JS, CSS) handled in SPAs, and how does it impact performance? **(Asset Loading)**
7. Why are SPAs less SEO-friendly by default? **(SEO Limitation)**
8. What are the strategies to make SPAs SEO-compatible? (e.g., SSR, pre-rendering, dynamic meta tags, which are handled in Next.js) **(SEO Optimization)**

🟢 Advanced Async / Practical Edge Cases

1. How to cancel previous API requests? **(AbortController)**
2. if a user click a button multiple time to fetch data how to cancel old api calls and use only the latest result ? **(Debounce / Cancel Requests)**
3. Does react use promises.allSettled() for parallel api calls how does that work internally ? **(Async Internals)**
